PROJETO INTEGRADOR — FASE 6 · FIAP

Sistema de Comunicação Interplanetária da Colônia (SCIC)

Relatório técnico — Colônia Aurora Siger

Equipe

CAETANO DE MEDEIROS BONA — RM569262
FERNANDO DOS SANTOS MOTTA — RM570046
JOEDSON DA SILVA SOUZA — RM573981
MYLENA RAMALHO DA SILVA TORQUATO — RM572383

Github: https://github.com/FIAP-404/atv-integradora-scic.git

Sumário

1. Introdução e contexto da solução
2. Dados simulados da Aurora Siger
3. Análise numérica e avaliação de erros
4. Modelo simples de previsão e avaliação de performance
5. Priorização de alertas com heap
6. Busca de registros com trie
7. Dispositivos, bases numéricas e eletricidade básica
8. Gerenciamento inteligente da comunicação
9. Reflexão social, cultural e sustentável
10. Limitações e possíveis melhorias
11. Conclusão
12. Evidências, reprodução e divisão do trabalho

1. Introdução e contexto da solução

A Aurora Siger é a colônia fictícia em torno da qual o projeto foi desenvolvido. O SCIC é um protótipo em Python que organiza dados operacionais e de comunicação **simulados** dos módulos da colônia, avalia a confiabilidade de uma previsão simples de latência, prioriza alertas críticos, permite consultas por prefixo e produz análises de apoio à decisão. O protótipo usa apenas a biblioteca padrão do Python, e as fórmulas foram escritas à mão para que cada passo possa ser explicado.

Todos os dados são sintéticos. A latência, em milissegundos, representa um atraso operacional didático dos módulos, e não o tempo de propagação de sinais entre planetas. Não há sensores, rede ou hardware reais: esses elementos aparecem de forma conceitual ou simulada.

1.1 Arquitetura do protótipo

Tabela 1 — Arquivos principais e responsáveis

Arquivo	Função	Responsável
`dados.py`, `gerar_dados.py`, `dados_aurora_siger.csv`	Leitura, validação e geração da base sintética	Fernando
`dispositivos.py`	Conversão entre bases numéricas e cálculo elétrico	Fernando
`codigo_fonte.py`	Menu no terminal e integração dos módulos	Fernando
`modulos/analise.py`	Indicadores, erros, modelo de previsão e métricas	Caetano
`modulos/alertas.py`	Heap de alertas, prioridade e gerenciamento inteligente	Joedson
`modulos/consultas.py`	Trie para busca por prefixo	Mylena
`tests/`	Testes automáticos de cada módulo	Cada integrante

1.2 Funcionalidades do menu

O programa é executado com `python3 codigo_fonte.py` e oferece um menu simples no terminal. O fluxo vai da leitura dos dados até a análise final:

- **1 — Carregar CSV:** troca a base por outro arquivo válido, sem alterar arquivos.
- **2 — Consultar registros:** filtra por trecho do nome do módulo e por ciclo.
- **3 — Bases e eletricidade:** mostra o código do sensor em decimal, binário e hexadecimal, e a potência P = V × I.
- **4 — Análise:** indicadores, divisão dos dados, modelo e métricas.
- **5 — Previsão:** estima a latência para uma carga de 0 a 100 %.
- **6 — Alertas (heap):** exibe o alerta mais urgente.
- **7 — Busca por prefixo (trie):** localiza registros por nome de módulo ou código hexadecimal.

2. Dados simulados da Aurora Siger

2.1 Descrição da base

A base `dados_aurora_siger.csv` tem **180 registros** de **seis módulos** (Habitação, Agricultura, Comunicação, Laboratório, Suporte Médico e Dados) ao longo de **30 ciclos**, um registro por módulo em cada ciclo. Os dados são organizados em arquivo CSV e carregados para listas de dicionários em Python.

Tabela 2 — Campos de cada registro

Campo	Tipo após a leitura	Domínio e unidade
`id`	inteiro	Único e positivo
`ciclo`	inteiro	Positivo; ordem temporal (1 a 30)
`modulo`, `tipo`	texto	Nome legível e classificação descritiva
`codigo_sensor`	inteiro	Decimal não negativo (160, 176, 192, 208, 224, 240)
`carga_pct`	decimal	0 a 100 %
`latencia_observada_ms`	decimal	Finito e não negativo, em ms
`tensao_v`, `corrente_a`	decimal	Finitos e não negativos, em V e A
`status`	texto	ativo, alerta ou manutencao
`mensagem`	texto	Descrição curta da situação simulada

A base não traz latência prevista, erros ou prioridades pré-calculados: esses resultados são produzidos pelos módulos de análise e de alertas.

2.2 Como os dados foram gerados

O gerador usa semente 36, o que permite reproduzir a mesma base. As regras criam cenários coerentes para a análise, e não um modelo de previsão treinado:

- Carga uniforme entre 15 e 95 %;
- Latência = 80 + 1,8 × carga + deslocamento do módulo + ruído normal (desvio de 5 ms) + pico de 70 ms quando (ciclo + 3 × índice do módulo) é múltiplo de 13;
- Deslocamentos de módulo, na ordem Habitação, Agricultura, Comunicação, Laboratório, Suporte Médico e Dados: 0, 8, −8, 4, −4 e 12 ms;
- Tensão de 12 V e corrente = 0,35 + 0,005 × carga, arredondada a quatro casas;
- Os picos recebem status `alerta`, e o módulo Dados no ciclo 17 recebe `manutencao` (manutenção programada).

2.3 Leitura e validação

A leitura com `csv.DictReader` valida o esquema exato de colunas, o número de colunas por linha, textos obrigatórios, id único e positivo, ciclo positivo, código não negativo, números finitos e não negativos, carga de no máximo 100 % e status conhecido. Entradas inválidas falham informando a linha e o motivo. A leitura **não imputa** valores e **não remove** anomalias em silêncio; latência e corrente iguais a zero são aceitas, o que exige tratamento

adequado nos cálculos. Os registros são ordenados por ciclo e id, e ao carregar outro arquivo pelo menu a base só é trocada depois de uma leitura bem-sucedida.

3. Análise numérica e avaliação de erros

3.1 Indicadores operacionais

Os indicadores **descrevem** a base; eles não fazem previsão. A função `analisar` os calcula sobre os 180 registros.

Tabela 3 — Indicadores calculados

Indicador	Valor	Leitura
Disponibilidade (status ativo)	91,67 % (165 de 180)	14 registros em alerta e 1 em manutenção
Latência média	189,263 ms (mín. 100,224; máx. 323,304)	Atraso operacional típico do enlace
Latência média por módulo	de 177,634 ms (Suporte Médico) a 201,170 ms (Laboratório)	O Laboratório tem a maior carga média (60,88 %) e a maior latência
Alertas por módulo	2 a 3 por módulo	Os picos estão espalhados; nenhum módulo concentra as falhas

3.2 Erro absoluto e erro relativo

Cada registro de saída mantém os campos de origem e acrescenta a latência prevista e dois erros:

- **Erro absoluto** = |observada − prevista|, em ms. Mostra o tamanho do erro.
- **Erro relativo** = erro absoluto ÷ **latência observada**, em fração (0,05 = 5 %). Permite comparar módulos e cargas de escalas diferentes. Se a observada for zero, o erro relativo é indefinido (`None`).

A latência observada é o valor medido, por isso é a referência. Os dois erros existem porque 8 ms pesam muito mais numa latência de 100 ms (8 %) do que numa de 300 ms (2,7 %).

Tabela 4 — Exemplos reais de erro

Registro	Observada (ms)	Prevista (ms)	Erro absoluto (ms)	Erro relativo
#145 Habitação, ciclo 25 (ativo)	134,275	134,587	0,312	0,2 %
#161 Suporte Médico, ciclo 27 (alerta)	217,411	141,890	75,521	34,7 %

3.3 Ponto flutuante e precisão numérica

O computador guarda números reais em binário com precisão finita: `0.1 + 0.2` resulta em `0.30000000000000004`. Para lidar com isso:

- Os coeficientes do modelo são arredondados a 4 casas **antes** de qualquer previsão, e previsões e erros a 3 casas, a mesma resolução da latência no CSV (0,001 ms). Mais casas seriam falsa precisão, e arredondar os coeficientes primeiro garante que `analisar` e `prever` deem o mesmo número;

- Comparações com zero usam tolerância (`math.isclose`) em vez de `== 0`, o que protege o denominador do erro relativo e o cálculo da reta;
- O R² fica `None` quando os valores observados não variam (divisão por zero);
- No módulo de eletricidade, os arredondamentos valem só para a apresentação, e a finitude dos resultados é verificada.

3.4 Quando um erro é aceitável ou preocupante

Em operação normal, a previsão erra em média cerca de 5,7 ms no conjunto de teste (cerca de 3 % da latência média de 189 ms), com RMSE de 7,7 ms. Um erro dessa ordem é **aceitável**: está no nível do ruído e do deslocamento por módulo que o modelo não representa.

Um erro **preocupante** é aquele bem acima disso. O módulo de alertas adota o limiar de **20 % acima do previsto**: só os picos chegam a esse patamar, e o ruído comum não. O registro #161, por exemplo, tem erro relativo de 34,7 % (denominador: observada) e está 53 % acima do previsto (denominador: prevista). As duas medidas respondem a perguntas diferentes, e cada módulo documenta a sua. Em uma colônia real, esse limiar teria de ser validado pelos responsáveis de cada módulo.

4. Modelo simples de previsão e avaliação de performance

4.1 Divisão dos dados e limpeza

Tabela 5 — Divisão temporal 60/20/20 dos ciclos

Conjunto	Ciclos	Registros	Uso
Treino	1–18	108	Calcular a reta
Validação	19–24	36	Decidir se a reta é melhor que uma linha de base
Teste	25–30	36	Medir o desempenho final, sem nenhuma decisão tomada a partir dele

A divisão é **temporal**: o modelo nunca vê ciclos futuros, como aconteceria na colônia. A validação influencia o modelo de propósito, porque escolhe entre a reta e a linha de base; o teste não. Um teste automatizado triplica as latências dos ciclos 25–30 e confere que o modelo e a seleção continuam idênticos e que só as métricas de teste mudam, ou seja, que não há vazamento do teste para o modelo.

Limpeza: a reta é ajustada só com registros `ativo` do treino. Os 9 registros de pico ou manutenção do treino ficam fora do ajuste, porque um pico soma cerca de 70 ms e puxaria a reta para cima. Esses registros **não são apagados**: recebem previsão e erro como os demais, e o erro alto é o que permite ao heap identificá-los. Nenhuma saída derivada (previsão ou erro) é usada como atributo de treino.

4.2 O modelo

Regressão linear simples por **mínimos quadrados**, a reta que minimiza a soma dos erros ao quadrado:

$$\text{inclinação} = \Sigma(x - \bar{x})(y - \bar{y}) \,/\, \Sigma(x - \bar{x})^2 \qquad \text{intercepto} = \bar{y} - \text{inclinação} \cdot \bar{x}$$

Resultado: **latência prevista = 1,7945 × carga + 82,0437 ms**. Cada ponto percentual de carga acrescenta cerca de 1,8 ms de atraso. Como os dados são sintéticos, o resultado pode ser conferido com a regra do gerador (80 + 1,8 × carga + deslocamento médio de +2 ms): o modelo recuperou ≈ 1,8 e ≈ 82 sem acesso a essa regra, o que indica que aprendeu a relação real e não o ruído.

Na validação, a reta foi comparada com uma linha de base ingênua que sempre prevê a média do treino: RMSE de **8,315 ms** contra **43,066 ms**. A reta foi escolhida. Se fosse pior que a média, o código usaria a média automaticamente.

A função `prever(carga_pct, resumo)` aplica a mesma fórmula e reproduz exatamente a latência prevista dos 180 registros. Ela rejeita cargas fora de 0–100 % ou não finitas. A faixa de carga do treino vai de **15,67 % a 94,95 %**; fora dela, como em 100 % (261,494 ms), a previsão é uma **extrapolação**, e nada garante que a relação continue linear com o sistema saturado.

4.3 Métricas de avaliação

Tabela 6 — Métricas por conjunto (nominal = só registros ativos)

Conjunto	Grupo	n	MAE (ms)	MSE (ms²)	RMSE (ms)	R²
Treino	nominal	99	7,873	85,476	9,245	0,9456
Treino	todos	108	12,478	436,873	20,902	0,7710
Validação	nominal	33	7,250	69,139	8,315	0,9617
Validação	todos	36	12,610	500,003	22,361	0,7586
Teste	nominal	33	5,737	58,726	7,663	0,9618
Teste	todos	36	11,173	479,273	21,892	0,7632

- **MAE** (erro absoluto médio): em operação normal, a previsão erra em média cerca de 6 ms no teste.
- **MSE** (erro quadrático médio): eleva os erros ao quadrado e, por isso, pune erros grandes; a unidade é ms², o que dificulta a leitura direta.
- **RMSE** (raiz do MSE): volta para ms e fica acima do MAE porque os erros grandes pesam mais. Uma diferença grande entre os dois indica erros maiores em alguns registros, como os picos.
- **R²**: fração da variação da latência explicada pela carga. O valor 0,96 indica que a carga explica cerca de 96 % da variação em operação normal.

4.4 Interpretação e limitações

- **Nenhum número isolado prova a qualidade do modelo.** O R² de 0,96 só tem valor porque se repete em treino, validação e teste (sem sobreajuste), porque a reta é cerca de cinco vezes melhor que a linha de base em RMSE e porque os coeficientes batem com a regra conhecida dos dados.
- **Picos:** com eles incluídos, o RMSE do teste sobe de 7,7 para 21,9 ms e o R² cai para 0,76. Isso não é defeito do modelo: a carga não explica um pico, e por isso um erro relativo alto funciona como sinal de anomalia, que alimenta a prioridade do heap.
- **RMSE ≈ 7,7 ms** é próximo do ruído do gerador (desvio de 5 ms) somado ao deslocamento por módulo, que o modelo ignora. Melhorar exigiria usar o módulo como entrada, mas o contrato `prever(carga, resumo)` recebe só a carga.
- **AIC, BIC, Grid Search e Random Search** não foram aplicados, porque o projeto não compara famílias de modelos nem ajusta hiperparâmetros: a escolha foi entre uma reta e uma linha de base.

5. Priorização de alertas com heap

Os resultados desta seção foram obtidos com a **análise real** do módulo de Caetano integrada ao módulo de alertas.

5.1 Como os alertas são representados

A função `priorizar(registros_analisados)` devolve uma fila de prioridade com três operações: `inserir`, `retirar` e `len`. Cada alerta é um **dicionário novo**, e os registros recebidos não são alterados. Ele guarda `id`, `modulo`, `ciclo`, `tipo`, `mensagem`, a `prioridade` (0 a 100), o `nivel`, a `essencialidade` e uma `justificativa` em texto. Os níveis são CRÍTICO (70 pontos ou mais), ALTO (a partir de 45), MÉDIO (a partir de 25) e BAIXO.

Um registro vira alerta quando atende a **pelo menos uma** condição: status `alerta` ou `manutencao`; latência observada mais de 20 % acima da prevista; ou tensão fora de 12 V ± 10 %. As duas últimas regras permitem que o monitoramento aponte problemas mesmo em módulos que ainda constam como ativos.

5.2 Critério de prioridade

Tabela 7 — Composição da nota de prioridade

Critério	Pontos	Por quê
Status	alerta = 40; manutenção = 15	O que já falhou vem antes do que está em manutenção
Essencialidade do módulo	suporte médico e comunicação = 25; habitação = 20; armazenamento = 15; agricultura = 10; laboratório = 5	Suporte de vida e enlace sustentam a colônia
Latência acima do previsto	até 20, proporcional ao desvio (50 % ou mais vale 20)	O desvio relativo compara escalas diferentes; o teto impede que o atraso sozinho passe à frente de tudo
Tensão fora de 12 V ± 10 %	10	Um problema elétrico pode derrubar o módulo
Ciclos aguardando	+1 por ciclo, até 5	Evita que um alerta antigo fique esquecido

O **desvio relativo** é (observada − prevista) ÷ prevista, com sinal, porque só um atraso acima do previsto deve aumentar a prioridade. O **desempate** segue esta ordem: prioridade, essencialidade, registro mais antigo e menor id.

5.3 Como o heap organiza os alertas

A estrutura é um **max-heap** guardado em um vetor, no qual o pai é sempre mais urgente que os filhos. Para a posição i, o filho esquerdo está em 2i+1, o direito em 2i+2 e o pai em (i−1)//2. Na **inserção**, o alerta entra no fim do vetor e sobe com **heapify-up** até a posição correta. Na **retirada**, a raiz sai, o último elemento ocupa o seu lugar e desce com **heapify-down**. Consultar o mais urgente custa O(1); inserir e retirar custam O(log n).

O módulo reproduz o exemplo da apostila: `[35, 12, 90, 4, 77, 51, 23]` vira `[90, 77, 51, 4, 12, 35, 23]` ao construir o max-heap.

5.4 Como o sistema seleciona o alerta mais urgente

Dos 180 registros, **15 viraram alertas** (14 com status alerta e 1 em manutenção). O mais urgente fica na raiz do heap: o **#117, Comunicação, ciclo 20, com 89,3 pontos**. A justificativa mostra a conta: status alerta (+40), módulo comunicação (+25), latência 48 % acima do previsto (+19,3) e 10 ciclos aguardando (+5).

Tabela 8 — Primeiras posições do vetor do heap

Posição	Pai	Nível	Pontos	Alerta
0	—	CRÍTICO	89,3	#117 Comunicação, ciclo 20
1	0	CRÍTICO	87,6	#5 Suporte Médico, ciclo 1
2	0	CRÍTICO	88,0	#161 Suporte Médico, ciclo 27
3	1	ALTO	65,4	#22 Laboratório, ciclo 4
4	1	CRÍTICO	79,1	#66 Dados, ciclo 11
5	2	CRÍTICO	80,6	#73 Habitação, ciclo 13
6	2	CRÍTICO	83,5	#39 Comunicação, ciclo 7
7	3	ALTO	62,0	#100 Laboratório, ciclo 17

O vetor não fica totalmente ordenado, porque o heap só garante que cada pai é mais urgente que os filhos (as posições 1 e 2, por exemplo, são irmãs). Mesmo assim, a raiz é sempre o mais urgente.

5.5 Vantagem sobre uma lista simples

Numa lista sem ordem, achar o mais urgente exige percorrer tudo: O(n) por atendimento e O(n²) no total. No heap, o total fica em O(n log n). As comparações contadas pelo próprio módulo confirmam isso:

Tabela 9 — Comparações para inserir n alertas e atender todos

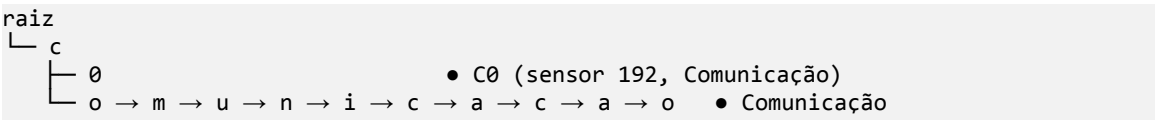
n alertas	Heap	Lista	Lista ÷ heap	Mesma ordem?
100	1.034	4.950	4,8×	sim
500	7.489	124.750	16,7×	sim
1.000	17.029	499.500	29,3×	sim
2.000	38.065	1.999.000	52,5×	sim

Com apenas 15 alertas a diferença é pequena. A vantagem do heap aparece quando o número de alertas cresce, como na tabela.

6. Busca de registros com trie

6.1 O que é uma trie e como funciona

Uma trie (árvore de prefixos) é uma árvore em que cada nó representa um caractere. O caminho da raiz até um nó forma um prefixo, e chaves que começam igual compartilham o mesmo caminho. Na base da Aurora Siger, por exemplo, `c0` (código hexadecimal do sensor da Comunicação) e `comunicacao` dividem o primeiro nó `c` e só se separam depois:



Para **inserir**, percorre-se a chave caractere por caractere a partir da raiz, criando só os nós que ainda não existem; no último nó guarda-se o id do registro. Para **buscar por prefixo**, desce-se pelo caminho do prefixo digitado. Se o caminho não existe, nenhum registro começa com aquele prefixo; se existe, coletam-se os ids de todos os nós abaixo dele.

6.2 O que é indexado

O módulo indexa duas colunas de cada registro: o nome do módulo e o código do sensor em hexadecimal, sem `0x`, gerado com a mesma conversão usada no módulo de dispositivos.

Tabela 10 — Chaves indexadas na trie

Módulo	Chave do nome (normalizada)	Código decimal	Chave hexadecimal
Habitação	habitacao	160	a0
Agricultura	agricultura	176	b0
Comunicação	comunicacao	192	c0
Laboratório	laboratorio	208	d0
Suporte Médico	suporte medico	224	e0
Dados	dados	240	f0

A **normalização** transforma o texto em minúsculas e remove os acentos. Por isso `COMUNICAÇÃO`, `Comunicação` e `comunicacao` geram a mesma chave, e `C0` equivale a `c0`.

6.3 Representação no código

- `NoTrie`: guarda um dicionário de filhos (caractere → nó) e um conjunto de ids dos registros cuja chave termina naquele nó.
- `Trie`: oferece `inserir(chave, id)`, `buscar(prefixo)` e `contar_nos()`.
- `construir_indice(registros)`: percorre os registros, valida os campos e insere o nome do módulo e o código hexadecimal de cada um.

Os resultados são **ids do campo {{id}} do CSV**, e não posições na lista. O conjunto de ids evita repetição: o registro #3 entra por duas chaves (`comunicacao` e `c0`) e aparece uma vez só. O retorno é ordenado, e o menu converte os ids de volta em registros, preservando a ordem de ciclo e id. Prefixo inexistente retorna lista vazia, e prefixo vazio retorna todos os ids.

6.4 Resultados da execução real

A base tem 180 registros e 12 chaves distintas (6 nomes e 6 códigos). A trie tem 70 nós, contra 73 caracteres somados das chaves: a diferença é o prefixo compartilhado.

Tabela 11 — Buscas de exemplo

Busca	Registros	Módulos encontrados
com	30	Comunicação
COMUNICAÇÃO	30	Comunicação (mesma chave de com)
lab	30	Laboratório
su	30	Suporte Médico
c0	30	Comunicação (pelo código hexadecimal)
d	60	Dados (nome) e Laboratório (código D0)
xyz	0	nenhum
(vazio)	180	todos

A busca `d` mostra um efeito a explicar ao usuário: um prefixo curto pode casar com um nome de módulo e com um código hexadecimal ao mesmo tempo, porque os dois tipos de chave estão na mesma árvore.

6.5 Por que a trie é adequada

A consulta por prefixo é exatamente o que a trie organiza: todas as chaves com o mesmo começo ficam sob o mesmo nó. Depois de digitar `co`, o sistema já está no nó certo, e `com` só continua a descida. Numa lista simples, cada consulta compara o prefixo com os 180 registros e o custo cresce com a base. Na trie, o custo para chegar ao nó do prefixo depende do tamanho do prefixo, e não da quantidade de registros; a coleta dos resultados custa proporcionalmente ao que é devolvido. Além disso, a trie agrupa: 12 chaves distintas apontam para 180 registros.

Vale ser honesto sobre a escala: com 180 registros e 12 chaves, a diferença de tempo em relação a uma lista é imperceptível. A vantagem é de organização e de crescimento (mais módulos, sensores e comandos); o protótipo a demonstra, mas não a mede.

6.6 Limitações

- A busca é por **início** da chave: `medico` não encontra Suporte Médico, porque o nome é indexado inteiro.
- Só o nome do módulo e o código hexadecimal são indexados; mensagens de alerta e outros campos não entram.
- A normalização cuida de acento e caixa, mas não corrige erros de digitação.
- A aplicação reconstrói a trie a cada busca. É aceitável nesta escala, mas uma versão maior guardaria o índice pronto.

7. Dispositivos, bases numéricas e eletricidade básica

7.1 Dispositivos de entrada e saída

Entrada. Sensores e medidores de carga, latência, tensão e corrente em cada módulo originariam os registros. No protótipo, o arquivo CSV é a entrada simulada desses medidores; nenhum sensor físico está conectado.

Saída. O terminal exibe o menu, as consultas e os resultados. Relatórios e arquivos de exemplo (como os da pasta `docs`) registram as execuções, e um dashboard poderia exibir os mesmos indicadores de forma contínua.

Interfaces de comunicação. Rede, USB, Wi-Fi e Bluetooth são citados apenas de forma conceitual, como possíveis meios de levar os dados dos sensores até o sistema. Não há dispositivo físico nem enlace real no protótipo.

7.2 Códigos de sensores em decimal, binário e hexadecimal

Cada módulo tem um código de sensor inteiro, guardado em decimal no CSV. O programa o converte para binário e hexadecimal com formatação do Python. Esses códigos são identificadores didáticos, e não endereços de rede reais.

Tabela 12 — Códigos dos sensores

Módulo	Decimal	Binário	Hexadecimal
Habitação	160	10100000	A0
Agricultura	176	10110000	B0
Comunicação	192	11000000	C0
Laboratório	208	11010000	D0
Suporte Médico	224	11100000	E0
Dados	240	11110000	F0

Exemplo de conversão. 192 = 12 × 16 + 0. Como 12 corresponde ao dígito C, o hexadecimal é **C0**; em binário, C = 1100 e 0 = 0000, o que dá **11000000**. O primeiro dígito hexadecimal (A a F) identifica o módulo, e o segundo é sempre 0. Esse mesmo código hexadecimal é uma das chaves da busca por trie (seção 6).

7.3 Tensão, corrente e potência

Com a tensão em volts e a corrente em ampères, a potência elétrica é P = V × I. No registro #1 (Habitação, ciclo 1), P = 12 V × 0,5565 A = **6,678 W**. Sob uma hipótese puramente resistiva, a lei de Ohm dá R = V ÷ I ≈ **21,5633 Ω**. Se a corrente for zero, a potência é zero e a resistência equivalente fica indefinida, sem divisão por zero.

Tabela 13 — Potência de cada módulo no ciclo 30 (tensão de 12 V)

Módulo	Carga (%)	Corrente (A)	Potência (W)
Habitação	76,01	0,7301	8,761
Agricultura	32,79	0,5140	6,168
Comunicação	66,55	0,6827	8,192

Módulo	Carga (%)	Corrente (A)	Potência (W)
Laboratório	90,92	0,8046	9,655
Suporte Médico	72,33	0,7116	8,539
Dados	47,39	0,5869	7,043
Total			48,36

> **Cuidados na interpretação.** A potência calculada é a potência **elétrica de entrada**, e não a potência radiada por um transmissor: perdas e eficiência não foram modeladas. Sem a duração dos ciclos em horas, também não se estima energia (Wh). A resistência é só uma hipótese equivalente, e não o modelo de um transmissor real

A soma de **48,36 W** no ciclo 30 é a demanda usada na análise de microrrede da seção 8.

8. Gerenciamento inteligente da comunicação

O SCIC segue, em escala didática, as três camadas de uma rede inteligente de comunicação: **coleta** (os registros dos sensores), **comunicação** (o enlace do módulo Comunicação) e **processamento** (o heap e as regras de gestão). A discussão abaixo parte dos resultados reais do protótipo.

8.1 Sensores e medidores inteligentes

A cada ciclo, cada um dos seis módulos entrega carga, latência, tensão e corrente. Numa colônia real, medidores inteligentes enviariam esses valores de forma contínua; no protótipo, o CSV com 30 ciclos faz esse papel. É esse fluxo de dados que alimenta todos os indicadores, erros e alertas das seções anteriores.

8.2 Monitoramento contínuo

A função de monitoramento lê os 30 ciclos em ordem e insere cada anomalia na fila de prioridade. Ao longo dos 30 ciclos, a fila chegou a **15 alertas**. O mais urgente começou como o #5 (Suporte Médico, ciclo 1, 87,6 pontos) e passou a ser o #117 (Comunicação, ciclo 20, 89,3 pontos), que continuou no topo até o ciclo 30.

O monitoramento também procura anomalias ainda não sinalizadas, ou seja, módulos com status ativo e latência mais de 20 % acima da prevista. Nesta base houve **0 detecções antecipadas**: todas as anomalias já vinham marcadas como alerta ou manutenção. A tensão é constante em 12 V, então a regra elétrica não disparou.

8.3 Automação com validação humana

A automação do protótipo consiste em ordenar os alertas, explicar a nota e sugerir uma ação. Ao atender o próximo alerta, o sistema mostra a justificativa e a ação sugerida, e pede a confirmação do operador. Para o #117, a sugestão é reagendar transmissões não urgentes e, se houver, comutar para o enlace redundante; para o #161, priorizar o canal da enfermaria e verificar a alimentação de 12 V. Se a ação é rejeitada, o alerta volta para a fila. A decisão final é sempre humana.

8.4 Armazenamento de dados e enlaces redundantes

O histórico de 180 registros permite comparar cada ciclo com os anteriores, e é ele que sustenta o monitoramento e a manutenção preditiva. Sobre o enlace de Comunicação, a análise mostra **2 ciclos com anomalia em 30**; no ciclo 30 ele está normal (194,5 ms, desvio de −3,4 %). A base tem um único enlace, e por isso o sistema recomenda justificar um **enlace redundante**: as anomalias no histórico mostram que o enlace pode falhar, e um enlace reserva permitiria comutar sem interromper a comunicação.

8.5 Manutenção preditiva

A análise acompanha o **desvio** da latência em relação à prevista, e não a latência bruta. Assim, uma variação normal de carga não é confundida com desgaste do módulo. O método tem três passos: calcular a taxa de variação do desvio por diferença finita; confirmar a tendência só se o desvio subir por 3 ciclos seguidos, mais de 2 pontos percentuais por

ciclo; e, confirmada a tendência, projetar pelo **método de Euler** até o desvio passar de 20 %.

Tabela 14 — Situação de cada módulo na manutenção preditiva

Módulo	Desvio atual	Classificação
Habitação	1,4 %	estável
Agricultura	5,4 %	estável
Comunicação	−3,4 %	estável
Laboratório	31,9 %	pico recente sem tendência: acompanhar
Suporte Médico	−5,4 %	estável
Dados	10,4 %	estável

Nenhum módulo tem tendência confirmada. O Laboratório aparece em acompanhamento por causa de um pico recente. Isso é coerente com a geração dos dados, que produz picos isolados e não desgaste progressivo; com dados reais, a mesma regra permitiria agendar a manutenção antes da falha.

8.6 Redes inteligentes e microrredes

A demanda elétrica no ciclo 30 é a soma de P = V × I dos seis módulos: **48,36 W**. Com 100 % da demanda disponível, o sistema opera em modo normal. Se a geração cair para 80 % (**38,69 W**), por exemplo numa tempestade de poeira que reduza a geração solar, o sistema entra em **modo de contingência**, à semelhança da operação ilhada de uma microrrede:

- Suporte Médico, Comunicação e Habitação são preservados.
- Um **segundo heap**, com a regra “menos essencial primeiro”, coloca em modo econômico (redução de 50 %) o Laboratório (−4,83 W), os Dados (−3,52 W) e a Agricultura (−3,08 W).
- A demanda cai para **36,93 W**, abaixo da capacidade disponível.

8.7 Síntese

Os indicadores de latência mostram o que é normal; o erro de previsão mostra o que foge do normal; o heap transforma esses desvios em uma ordem de atendimento; a manutenção preditiva e a análise do enlace indicam o que pode falhar; e a microrrede mostra como decidir quando falta energia. Em conjunto, esses resultados ilustram como sensores, monitoramento contínuo, automação, redundância, manutenção preditiva e microrredes se encaixam na gestão da comunicação da Aurora Siger.

9. Reflexão social, cultural e sustentável

O SCIC é um protótipo com dados simulados, mas as decisões que ele organiza (qual alerta atender primeiro, qual módulo reduzir quando falta energia) seriam tomadas por pessoas e afetariam uma comunidade. Esta seção discute cinco pontos, sempre a partir do que o protótipo faz.

9.1 Uso eficiente da comunicação e sustentabilidade

Numa colônia, energia e atenção são recursos finitos. O heap de alertas ajuda a gastar atenção onde há mais risco: o alerta mais urgente sai primeiro, em vez de a equipe percorrer tudo. O balanço de microrrede mostra o lado da energia: no ciclo 30, a demanda soma 48,36 W e, com 80 % disponível, o sistema preserva Suporte Médico, Comunicação e Habitação e coloca Laboratório, Agricultura e Dados em modo econômico, o que leva a demanda a 36,93 W. A ideia é cortar o desperdício antes de cortar o essencial.

Duas ressalvas: esses números vêm de uma simulação com hipóteses simples (por exemplo, redução fixa de 50 %), e a trie, em si, não economiza energia de forma mensurável nesta escala. A contribuição para a sustentabilidade está nas decisões de priorização, e não na busca.

9.2 Conhecimentos tradicionais indígenas e decisões sobre recursos

Há muitos povos indígenas no Brasil, com saberes distintos, então não cabe generalizar. Em termos gerais, muitas dessas tradições tratam o território como algo a ser cuidado, e não apenas explorado, e valorizam usar o necessário e decidir pensando na coletividade e no longo prazo. Essa é uma inspiração para o desenho do sistema, e não uma regra técnica.

O protótipo oferece um ponto concreto para aplicá-la. Na tabela de essencialidade, a Agricultura vale 10 pontos e o Laboratório 5, e na contingência esses módulos estão entre os primeiros a entrar em modo econômico. Se a Agricultura representa a produção de alimento da colônia, considerá-la menos essencial que a comunicação é uma decisão que merece revisão por quem conhece o cultivo. Como os pesos foram definidos pela própria equipe, a recomendação é validá-los com os responsáveis pelos módulos e torná-los fáceis de revisar.

9.3 Diversidade cultural e sistemas não excludentes

Um sistema pode excluir por detalhes pequenos. Na busca, a normalização de acentos e caixa existe para que `Médico`, `MEDICO` e `medico` cheguem ao mesmo resultado: quem usa teclado sem acento ou digita de modos diferentes não fica de fora, e quem prefere códigos pode usar o hexadecimal em vez do nome.

O protótipo tem limites: a interface é só em português e em terminal, e a base não contém dados pessoais nem classifica pessoas. Um risco mais sutil é o viés de critério: os pesos de prioridade e os limiares foram escolhidos por poucas pessoas, com dados simulados. Numa colônia real, essas definições refletiriam um único ponto de vista, a menos que fossem discutidas com representantes dos módulos e da comunidade. As mensagens de alerta devem se manter técnicas e neutras, como as do CSV (por exemplo, “Pico simulado de latência”), sem linguagem que culpe ou rotule pessoas ou grupos.

9.4 Transparência nas decisões baseadas em dados

Uma decisão automatizada só pode ser questionada se puder ser explicada. Cada alerta do heap carrega uma justificativa em texto que mostra de onde vieram os pontos (por exemplo, status alerta +40 e módulo comunicação +25, seguidos dos pontos de latência e de espera). A busca por trie devolve ids que levam direto aos registros do CSV, então qualquer resultado pode ser conferido na base. Os dados são declarados como simulados, as seções técnicas listam suas limitações e o código é comentado. A transparência também exige não esconder o que o sistema não sabe, como a diferença entre dado simulado e medição real.

9.5 A equipe humana continua responsável

O protótipo recomenda, mas não age sozinho. O módulo de alertas mostra a justificativa e a ação sugerida e pede a confirmação do operador; se a ação é rejeitada, o alerta volta para a fila. Na opção 6 do menu, o sistema apenas exibe o alerta mais urgente, e em nenhum ponto do protótipo uma ação é executada automaticamente. Isso importa porque uma automação sem supervisão poderia reduzir um módulo que, naquele momento, é crítico, ou repetir um viés dos pesos sem que ninguém perceba. A trie, por sua vez, é apenas uma ferramenta de consulta: quem decide é a equipe.

9.6 Síntese

O SCIC apoia o uso mais eficiente da comunicação ao priorizar alertas pelo risco e ao propor um modo de contingência que corta primeiro o que é menos essencial, reduzindo desperdício. Para que isso seja responsável, os critérios precisam ser revisados e discutidos com quem conhece cada módulo, as decisões precisam ser explicadas e a palavra final precisa ser humana. A busca por trie contribui para a inclusão ao aceitar variações de acento e caixa e ao permitir consultar tanto por nome quanto por código.

10. Limitações e possíveis melhorias

10.1 Limitações

- **Dados sintéticos.** A base é simulada e a latência é um atraso didático, que não modela propagação física entre planetas. Resultados com dados reais exigiriam nova validação.
- **Modelo.** Usa só a carga como entrada, ignora o módulo e extrapola fora da faixa de 15,67 % a 94,95 %. Com os picos incluídos, o R^2 cai de 0,96 para 0,76.
- **Prioridade.** Os pesos e os limiares foram definidos pela equipe e deveriam ser validados com os responsáveis pelos módulos. A fila contém todo o histórico de alertas, então o critério de espera satura em 5 pontos para alertas antigos.
- **Gestão inteligente.** A projeção de Euler supõe taxa constante, a base tem um único enlace de comunicação e a redução para modo econômico é fixa em 50 %.
- **Trie.** A busca é só por início de chave, indexa apenas nome de módulo e código hexadecimal, e o índice é reconstruído a cada consulta.
- **Eletricidade.** A potência é elétrica de entrada, a resistência é uma hipótese resistiva, e sem a duração dos ciclos não se calcula energia.
- **Menu.** A opção 6 apenas exibe o alerta mais urgente; o fluxo com confirmação do operador existe no módulo, mas não é chamado pelo menu.

10.2 Possíveis melhorias

- Usar o módulo como variável do modelo (ou um modelo por módulo) e comparar modelos com AIC, BIC e Grid Search quando houver mais dados.
- Incluir um segundo enlace na base simulada para testar a comutação para redundância.
- Guardar o índice da trie pronto e permitir busca por palavra dentro do nome.
- Tornar os pesos de prioridade configuráveis e validados, e fazer a opção 6 do menu chamar o fluxo com confirmação do operador.
- Registrar a duração dos ciclos para estimar energia (Wh) e não só potência.
- Gerar gráficos com Matplotlib para apresentar latência, erros e alertas ao longo dos ciclos.

11. Conclusão

O SCIC cumpre o papel de protótipo: carrega e valida uma base simulada da Aurora Siger, calcula indicadores e erros, estima a latência com um modelo simples avaliado por MAE, MSE, RMSE e R^2, prioriza alertas com heap, localiza registros por prefixo com trie e relaciona tudo isso a dispositivos, bases numéricas, eletricidade básica e gestão inteligente da comunicação. O resultado mais importante não é um número isolado, mas a forma como os números se conectam: o erro de previsão aponta anomalias, o heap define a ordem de atendimento, a microrrede orienta cortes de energia e a equipe humana valida as decisões. As limitações listadas indicam onde um sistema real precisaria de dados reais, critérios validados e supervisão contínua.

12. Evidências, reprodução e divisão do trabalho

12.1 Como executar e testar

O projeto requer Python 3.10 ou superior e usa apenas a biblioteca padrão. Na pasta do projeto:

```
python3 codigo_fonte.py                      # menu no terminal
python3 codigo_fonte.py --demo               # demonstração sem menu
python3 -m unittest discover -s tests -v     # todos os testes
```

A suíte completa tem **60 testes**, e todos passaram na execução que originou este relatório.

Tabela 15 — Testes por integrante

Arquivo de teste	Testes	Cobertura principal
`tests/test_fernando.py`	13	Leitura e validação dos dados, bases numéricas, eletricidade, menu e CLI
`tests/test_caetano.py`	24	Indicadores, erros, divisão sem vazamento, modelo, métricas e previsão
`tests/test_joedson.py`	13	Prioridade, heap, desempate e gestão inteligente
`tests/test_mylena.py`	10	Normalização, trie, busca por módulo e código hexadecimal, integração

12.2 Divisão do trabalho

Tabela 16 — Entregas por integrante

Integrante	Entregas
Fernando	Base de dados, geração sintética, leitura e validação, menu e integração, entrada e saída, bases numéricas e cálculo elétrico
Caetano	Indicadores, erros absoluto e relativo, ponto flutuante, modelo simples, divisão dos dados, métricas e interpretação
Joedson	Alertas e prioridade, heap, desempates e discussão de gestão inteligente conectada aos dados
Mylena	Trie e busca por prefixo, reflexão social, cultural e sustentável, e organização do relatório